Lab no 9

Lab Task 1

Unique Index:

```
CREATE UNIQUE INDEX idx_room_number ON Rooms(room_number);
```

Composite Index

```
CREATE NONCLUSTERED INDEX idx_price_availability ON Rooms(price, is_available);
SELECT * FROM Rooms WHERE price BETWEEN 4000 AND 9000 AND is_available = 1;
```

Full-Text Index

```
ALTER TABLE Rooms ADD description NVARCHAR(200);
UPDATE Rooms
SET description = 'Luxury sea-facing room with breakfast';
CREATE FULLTEXT CATALOG RoomCatalog AS DEFAULT;
CREATE FULLTEXT INDEX ON Rooms(description)
KEY INDEX PK__Rooms__3214EC07; -- Use actual PK name
SELECT * FROM Rooms WHERE CONTAINS(description, 'sea');
```

Spatial Index

```
ALTER TABLE Hotels ADD location GEOGRAPHY;
UPDATE Hotels
SET location = GEOGRAPHY::Point(24.8607, 67.0011, 4326) WHERE hotel_id = 1;

CREATE SPATIAL INDEX idx_hotel_location ON Hotels(location);
```

Function-Based Index

```
CREATE INDEX idx_lower_city ON Hotels(LOWER(city));
SELECT * FROM Hotels WHERE LOWER(city) = 'karachi';
```

Lab Task 2

Create a Simple View

A simple view is based on one table.

```
47 •    CREATE VIEW View_ProductDetails AS
48      SELECT ProductName, Category, Price
49      FROM Products;
50
51      -- Test the view
52 •    SELECT * FROM View_ProductDetails;
53
54
```

Result Grid | Filter Rows: | Expor

ProductName	Category	Price
Smartphone	Electronics	699.00
Laptop	Electronics	1299.00
Coffee Maker	Appliances	89.00
Book	Books	15.00
Headphones	Electronics	199.00

Create a Complex View

A **complex view** can involve **joins and calculations**.

```
CREATE VIEW View_OrderSummary AS
SELECT
    c.CustomerName,
    p.ProductName,
    o.Quantity,
    (o.Quantity * p.Price) AS TotalPrice,
    o.OrderDate
FROM Orders o
JOIN Customers c ON o.CustomerID = c.CustomerID
JOIN Products p ON o.ProductID = p.ProductID;

-- Test the view
SELECT * FROM View_OrderSummary;
```

Create an Updatable View

A view on a single table without aggregation can be updated.

```
68 •    UPDATE View_Stock
69      SET Stock = 12
70      WHERE ProductID = 1;
71
72      -- Verify changes in Products table
73 •    SELECT * FROM Products;
74
75
76
```

Result Grid | Filter Rows: | Edit:

ProductID	ProductName	Category	Price	Stock
1	Smartphone	Electronics	699.00	12
2	Laptop	Electronics	1299.00	20
3	Coffee Maker	Appliances	89.00	30
4	Book	Books	15.00	100
5	Headphones	Electronics	199.00	40

Inline View

An inline view is just a subquery inside a SELECT. It's temporary, no permanent storage.

```
73 •    SELECT *
74      FROM (
75          SELECT Category, AVG(Price) AS AvgPrice
76          FROM Products
77          GROUP BY Category
78      ) AS InlineAvg
79      WHERE AvgPrice > 10000;
80
```

Result Grid | Filter Rows: | Export: | Wrap Ce

Category	AvgPrice

Materialized / Indexed View

```sql
-- Create table for materialized view
CREATE TABLE Materialized_SalesReport AS
SELECT
    p.Category,
    COUNT(*) AS TotalOrders,
    SUM(o.Quantity) AS TotalQuantity,
    SUM(o.Quantity * p.Price) AS TotalRevenue
FROM Orders o
JOIN Products p ON o.ProductID = p.ProductID
GROUP BY p.Category;

-- Query materialized table
SELECT * FROM Materialized_SalesReport;
```

Lab Task 3

Indexes

Non-Clustered Index

```
CREATE INDEX idx_product_category ON Products(category);
CREATE INDEX idx_customer_city ON Customers(city);
```

Unique Index

```
CREATE UNIQUE INDEX idx_email ON Customers(email);
```

Composite Index

```
CREATE INDEX idx_price_stock ON Products(price, stock);
```

Full-Text Index

```
CREATE FULLTEXT INDEX idx_product_name_ft ON Products(product_name);
-- Test: Search product containing 'Laptop'
SELECT * FROM Products WHERE MATCH(product_name) AGAINST('Laptop');
```

Function-Based Index

```
CREATE INDEX idx_lower_product_name ON Products((LOWER(product_name)));
-- Test: Case-insensitive search
SELECT * FROM Products WHERE LOWER(product_name) = 'mobile';
```

Drop Index

```
DROP INDEX idx_price_stock ON Products;
-- Observe query performance for price+stock queries
```

Views

Simple View

```sql
CREATE VIEW View_Customers AS
SELECT name, email, city FROM Customers;

-- Test
SELECT * FROM View_Customers;
```

Complex View

```sql
CREATE VIEW View_OrderSummary AS
SELECT
    c.name AS CustomerName,
    p.product_name AS ProductName,
    o.quantity,
    (o.quantity * p.price) AS TotalPrice,
    o.order_date
FROM Orders o
JOIN Customers c ON o.customer_id = c.customer_id
JOIN Products p ON o.product_id = p.product_id;

-- Test
SELECT * FROM View_OrderSummary;
```

Updatable View

```sql
CREATE VIEW View_ProductStock AS
SELECT product_id, product_name, stock
FROM Products;

-- Update stock via view
UPDATE View_ProductStock SET stock = 10 WHERE product_id = 1;

-- Test
SELECT * FROM Products WHERE product_id = 1;
```

Conditional View

```sql
CREATE VIEW View_AvailableProducts AS
SELECT product_id, product_name, price, stock
FROM Products
WHERE stock > 0 AND price < 10000;

-- Test
SELECT * FROM View_AvailableProducts;
```

Materialized / Indexed View

```sql
CREATE TABLE Materialized_Revenue AS
SELECT
    p.product_name,
    SUM(o.quantity) AS TotalSold,
    SUM(o.quantity * p.price) AS TotalRevenue
FROM Orders o
JOIN Products p ON o.product_id = p.product_id
GROUP BY p.product_name;

-- Test
SELECT * FROM Materialized_Revenue;
```

Update, Delete, Modify Views

Update stock via updatable view:

```sql
UPDATE View_ProductStock SET stock = 5 WHERE product_id = 2;
SELECT * FROM Products WHERE product_id = 2;
```

Delete record via view

```sql
DELETE FROM View_ProductStock WHERE product_id = 3;
SELECT * FROM Products;
```

Modify a view

```sql
ALTER VIEW View_AvailableProducts AS
SELECT product_id, product_name, price, stock
FROM Products
WHERE stock > 0 AND price < 15000;

SELECT * FROM View_AvailableProducts;
```

Drop unnecessary views:

```sql
DROP VIEW IF EXISTS View_Customers;
DROP VIEW IF EXISTS View_OrderSummary;
DROP VIEW IF EXISTS View_ProductStock;
DROP VIEW IF EXISTS View_AvailableProducts;
DROP VIEW IF EXISTS View_AllCustomers;
```